

Kizzfinder: Layerwise Kalman-Style Gradient Trust Scaling

1 Motivation

Bayes by Backprop makes neural network training uncertainty-aware by learning a distribution over each weight instead of a single fixed value. This allows the model to represent parameter uncertainty and reduce overconfident predictions. However, the optimizer still receives gradients from noisy mini-batches and randomly sampled weights, so the training signal itself can fluctuate.

Our Kalman-inspired trust step adds uncertainty awareness to the optimization process. Instead of only asking how uncertain each weight is, the method asks how trustworthy each layer’s current gradient is. For each layer, we compare the current gradient norm to a running average. If the gradient behavior is stable, the update is mostly trusted. If the gradient behavior is unstable, the update is dampened before Adam applies it.

2 Algorithm Overview

The normal training pipeline is:

$$\text{loss} \rightarrow \text{backprop gradient} \rightarrow \text{Adam update}$$

Kizzfinder inserts a trust-scaling step between backpropagation and Adam:

$$\text{loss} \rightarrow \text{backprop gradient} \rightarrow \text{Kalman-style trust scaling} \rightarrow \text{Adam update}$$

For each layer l , the method tracks the gradient norm:

$$z_l = \|g_l\|_2$$

Then it updates a running average gradient norm:

$$\bar{g}_l \leftarrow \beta \bar{g}_l + (1 - \beta) z_l$$

Next, it estimates how noisy the current gradient is relative to recent behavior:

$$R_l \leftarrow \beta R_l + (1 - \beta)(z_l - \bar{g}_l)^2$$

The Kalman-style trust coefficient is:

$$K_l = \frac{P_l}{P_l + R_l}$$

If R_l is small, then $K_l \approx 1$, so the gradient is mostly trusted. If R_l is large, then $K_l \approx 0$, so the gradient is strongly dampened.

The filtered gradient is:

$$g_l \leftarrow K_l g_l$$

Adam then updates the model using the scaled gradient rather than the raw gradient.

3 Step-by-Step Description

- Compute the loss for the current mini-batch.
- Backpropagate to obtain each layer's gradient, g_l .
- Compute each layer's gradient norm:

$$z_l = \|g_l\|_2$$

- Update the running average gradient norm:

$$\bar{g}_l \leftarrow \beta \bar{g}_l + (1 - \beta) z_l$$

- Estimate gradient noise by comparing the current norm to the running average:

$$R_l \leftarrow \beta R_l + (1 - \beta)(z_l - \bar{g}_l)^2$$

- Compute the Kalman-style trust factor:

$$K_l = \frac{P_l}{P_l + R_l}$$

- Scale the layer gradient:

$$g_l \leftarrow K_l g_l$$

- Pass the filtered gradient to Adam.
- Adam updates the model weights using the dampened gradient.

4 Pseudocode

Algorithm 1 Kizzfinder: Layerwise Kalman-Style Gradient Trust Scaling

Require: Layer gradients $\{g_l\}_{l=1}^L$, optimizer, smoothing factor β , uncertainty states $\{P_l\}$, running gradient norms $\{\bar{g}_l\}$, running noise estimates $\{R_l\}$

1: **for** each training step **do**

2: Compute loss $\mathcal{L}$

3: Backpropagate to obtain gradients $\{g_l\}_{l=1}^L$

4: **for** each layer $l = 1, \dots, L$ **do**

5: Compute gradient norm:

$$z_l \leftarrow \|g_l\|_2$$

6: Update running gradient norm:

$$\bar{g}_l \leftarrow \beta \bar{g}_l + (1 - \beta) z_l$$

7: Update running gradient-noise estimate:

$$R_l \leftarrow \beta R_l + (1 - \beta)(z_l - \bar{g}_l)^2$$

8: Compute Kalman-style trust coefficient:

$$K_l \leftarrow \frac{P_l}{P_l + R_l}$$

9: Scale layer gradient:

$$g_l \leftarrow K_l g_l$$

10: **end for**

11: Apply optimizer update using scaled gradients

12: **end for**

5 Interpretation

Kizzfinder is not a full Kalman filter over the neural network parameters. A full Kalman filter would require tracking much larger uncertainty structures, which would be too expensive for deep networks. Instead, this method uses the central Kalman idea in a lightweight way: noisy measurements should be trusted less than stable measurements.

Here, the “measurement” is the layer’s gradient norm. Stable gradient norms suggest reliable updates, while unstable gradient norms suggest noisy training evidence. Therefore, Kizzfinder acts as a Kalman-inspired adaptive gradient damping method that makes training more cautious when layerwise gradients appear unreliable.